You will have to choose 256 colors and “bin” each of the 16M original colors into these 256 buckets; the key is how you choose the destination colors scale. There are, of course, many ways to do that, you could choose the same 256 colors for all images, perhaps by sampling uniformly the original 16M scale; but you could also decide for each image what’s the best scale to reduce the loss of information.

How you do choose this scale was actually the subject of an interview question I was asked by one of the best engineers with whom I had the pleasure to work: we’ll leave it as an exercise for the reader, thinking about how this problem can be best solved, so we avoid spoiling the question.

But once you somehow came up with the best choice for the 256 colors to use, how do you transform each pixel from one scale to the other?

Here it is where nearest neighbor search comes into play: we create a k-d tree, or S-tree, containing each of the 256 selected colors for the destination scale; the dimension of the search space will be, as you can imagine, 3.

Listing 11.9 Down-sampling image colors using NN-search

```
tree ← S-tree(sampledColors,
3)                                          #1
for pixel in sourceImage
                                       #2
  (r, g, b) ←
pixel.color                                                  #3
  sampled_color ← tree.nearestNeighborSearch(r, g,
b)                #4
```